

Security in Communications (including Messaging Apps)

NECKER

December 5, 2025

Contents

1	MAC	2
1.1	MAC: Media Access Control Address (Networking)	2
1.2	MAC: Message Authentication Code (Security)	2
2	Operating Principle of the MAC	3
2.1	Tag Generation: $t' = M_k(m')$ (Sender Side)	3
2.2	Tag Verification: $t'' \stackrel{?}{=} t'$ (Recipient Side)	3
3	Cryptography and OSI Model Layers	5
4	Security Protocols: IPsec, HTTPS, and TLS	7
4.1	IPsec (Internet Protocol Security) - Layer 3 (Network)	7
4.2	TLS (Transport Layer Security) - Layer 4/5 (Transport)	8
4.3	HTTPS (Hypertext Transfer Protocol Secure) - Layer 7 (Application)	9
5	End-to-End Encryption and Key Exchange	10
5.1	End-to-End Encryption (E2EE)	10
5.2	Diffie-Hellman (DH) Key Exchange Algorithm	10
6	Modern Protocols for Secure Messaging	12
6.1	1. MTPProto (Mobile Telegram Protocol)	12
6.2	2. Signal Protocol (Formerly TextSecure)	12
7	Comparative Analysis of Messaging Platforms	13
7.1	Protocols and Encryption (Confidentiality Focus)	13
7.2	Privacy and Metadata (Privacy Focus)	13
7.3	Confidentiality Analysis: Practical Limits of E2EE and Functional Backdoors	13

1 MAC

The acronym MAC identifies two fundamental but completely distinct concepts in computer science and cryptography: the **Media Access Control Address** and the **Message Authentication Code**.

1.1 MAC: Media Access Control Address (Networking)

The MAC Address is a **hardware identifier** that operates locally within networks.

- **Nature:** It is a permanent and unique **physical address**, hardcoded into the network interface card (NIC) by the manufacturer.
- **Function:** To uniquely identify a device within a local area network (LAN), allowing data packets to reach the correct device (Layer 2 of the OSI model).
- **Format:** It is a fixed sequence of 48 bits (6 bytes), represented in hexadecimal format (e.g., 00:1A:2B:3C:4D:5E).
- **Network Layer:** It operates at the Data Link layer (Layer 2 of the OSI model).
- **Example of use:** Used by the ARP (Address Resolution Protocol) and Ethernet protocols.

1.2 MAC: Message Authentication Code (Security)

The MAC in cryptography is a **security mechanism** aimed at verifying data in transit. ([covered in this document](#))

- **Nature:** It is a dynamic **cryptographic code**, generated for each individual message.
- **Function:** To guarantee the **integrity** (that the message has not been altered) and the **authenticity** (that it originates from whoever possesses the secret key) of the message, **but not its secrecy**.
- **Input and Output:** Its generation requires the message (m) and a **symmetric secret key** (k), producing a tag (t).

$$t = M_k(m)$$

- **Network Layer:** It operates at the application or transport layer (for example, in TLS/SSL).
- **Example of use:** HMAC (Keyed-Hash Message Authentication Code), CMAC.

2 Operating Principle of the MAC

The **Message Authentication Code (MAC)** is a cryptographic function that ensures the **integrity** and **authenticity** of a message m using a symmetric secret key k . The MAC algorithm is denoted by M .

2.1 Tag Generation: $t' = M_k(m')$ (Sender Side)

The Tag generation phase is executed by the sender and aims to create a unique authentication code for the message.

1. **Input:** The sender takes as input the **original message** (m') of arbitrary length and the **symmetric secret key** (k), which must be known only to the sender and the recipient.
2. **Processing:** The MAC algorithm (M) is applied to m' and k . The MAC is not a simple hash function, but a **cryptographic function** that depends heavily on the key.
3. **Output:** The result of the processing is the **Tag** (t'), a fixed-length bit sequence (e.g., 128 bits), calculated as:

$$t' = M_k(m')$$

4. **Transmission:** The sender transmits the pair consisting of the **original message** (m') and the **Tag** (t') to the recipient.

The unforgeability of the MAC ensures that an attacker not in possession of k cannot compute a valid tag for a modified message.

2.2 Tag Verification: $t'' \stackrel{?}{=} t'$ (Recipient Side)

The verification phase is executed by the recipient to confirm the integrity and authenticity of the received message.

1. **Reception:** The recipient receives the potential message (m') and the received tag (t').
2. **Tag Recalculation:** Using the **same secret key** (k) and the **same MAC algorithm** (M), the recipient recalculates the tag on the received message (m'), obtaining the verification tag (t''):

$$t'' = M_k(m')$$

3. **Comparison:** The recipient rigorously compares the calculated tag (t'') with the received tag (t').

$$\text{Evaluation: } t'' \stackrel{?}{=} t'$$

4. Outcome:

- If $t'' = t'$, the message is **accepted**. Integrity is confirmed (the message has not been altered) and authenticity is guaranteed (it was generated by someone holding k).
- If $t'' \neq t'$, the message is **rejected**. This indicates that the message was modified (violation of integrity) or was sent by an unauthorized source (violation of authenticity).

MAC Flow Diagram

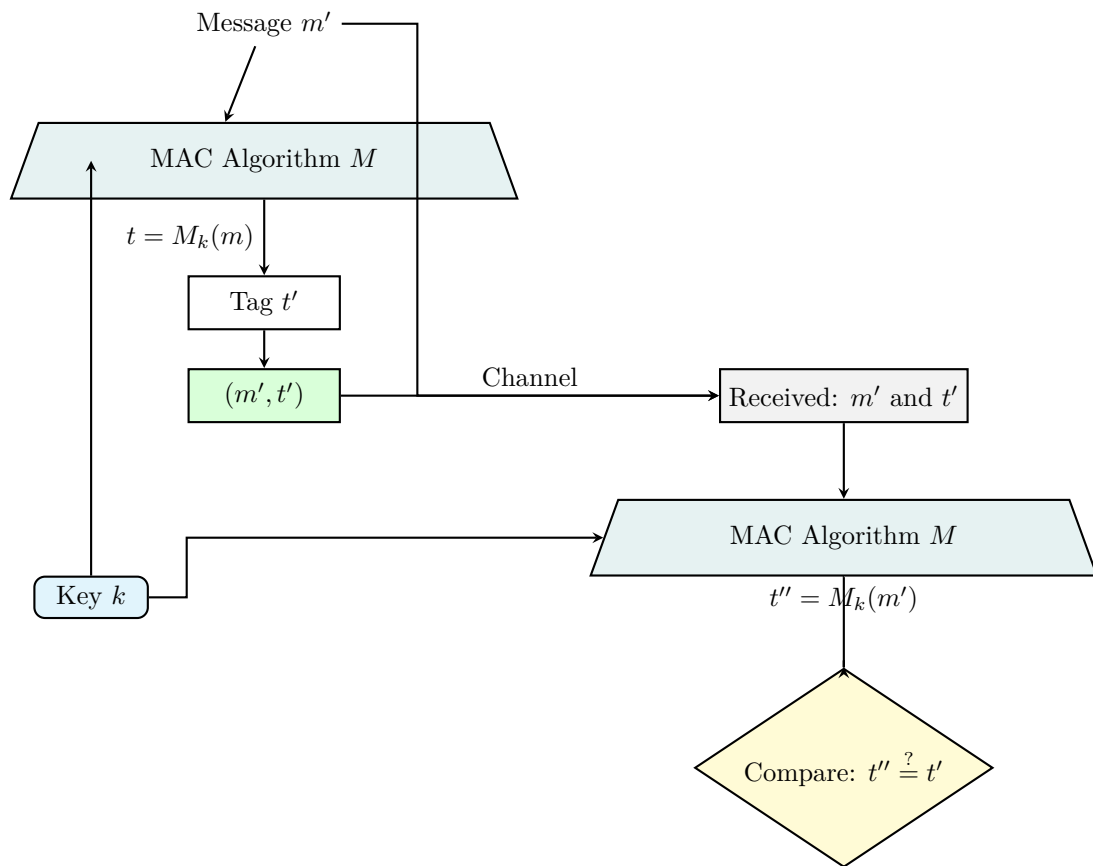


Figure 1: Generation and Verification Flow of a Message Authentication Code (MAC).

3 Cryptography and OSI Model Layers

Having introduced the **MAC**, to guarantee **confidentiality** we introduce encryption, which can take place at different levels of the OSI (Open Systems Interconnection) model. The choice of layer determines the **scope of protection** and which packet **headers** are encrypted.

Layer 7: Application Layer

This is the layer where encryption is applied directly to the **user payload** data before it is passed down to lower layers.

- **What is encrypted:** **Only the content of the message** or the body of the file. Application headers (e.g., an email subject line) may remain in cleartext.
- **Advantages:** Confidentiality is maintained **end-to-end**, even if the data transits through untrusted intermediate nodes. Ideal for email (PGP, S/MIME).
- **Examples:** PGP (Pretty Good Privacy) or GnuPG for emails.

Layer 6/5: Presentation/Session Layer

These layers are often handled in combination for session-level encryption.

- **What is encrypted:** **Application data** and partially some **session/presentation information**.
- **Advantages:** Protects the entire communication session. It is transparent to the applications above it and to the networks below it.
- **Examples:** SSL/TLS (Secure Sockets Layer/Transport Layer Security) is not strictly Layer 6, but its logical implementation is often positioned here or right above Layer 4.

Layer 4: Transport Layer

This layer protects communication between the **two endpoints** (client and server).

- **What is encrypted:** The **entire data segment** (payload) sent between two hosts. Network headers (IP) and transport headers (TCP/UDP) remain in cleartext.
- **Advantages:** Provides confidentiality and integrity between the two endpoint applications. Intermediate routers cannot read the data but can still route it normally.
- **Examples:** TLS/SSL (its practical implementation is frequently seen here), SSH (Secure Shell).

Layer 3: Network Layer

This is the level of encryption that protects the **IP datagram** (excluding routing information).

- **What is encrypted:** The **entire data packet** (including the payload and Layer 4 headers), **except the external IP headers** (which are necessary for routing).
- **Advantages:** Provides protection on a **hop-by-hop** or **tunneling** basis between two network gateways. It is transparent to both the user and the applications.
- **Examples:** IPsec (Internet Protocol Security), which can operate in transport or tunnel mode.

Layer 2: Data Link Layer

Encryption here focuses on protecting traffic between two directly connected nodes (e.g., a PC and a router or two nodes within a LAN).

- **What is encrypted: The whole frame** (complete data packet, including IP and TCP headers), **except the physical header** (needed at Layer 1 for transmission).
- **Advantages:** Protection on a **hop-by-hop** basis (between two physically connected devices).
- **Examples:** **MACsec** (Media Access Control Security), **WPA2/3** for Wi-Fi networks (which operate across Layer 2 and Layer 1).

4 Security Protocols: IPsec, HTTPS, and TLS

Security protocols operate at different levels of the network stack to ensure confidentiality, integrity, and authenticity. IPsec operates at the Network Layer, while HTTPS relies on TLS, which operates between the Transport and Application layers.

4.1 IPsec (Internet Protocol Security) - Layer 3 (Network)

IPsec is a **standardized protocol suite** that extends the Layer 3 Internet Protocol (IP), adding essential security features directly into the network layer. Its strength lies in providing **transparent** protection to all protocols and applications sitting above the IP layer.

Components and Functions

IPsec relies on two key protocols to define its security services:

1. AH (Authentication Header) - Integrity and Authenticity:

- **Purpose:** Guarantees the **integrity** (verifies that the packet has not been modified) and **authenticity** (verifies the packet source) of the IP packet.
- **Mechanism:** Calculates a **MAC** (Message Authentication Code) over the payload and most of the IP header. Mutable fields in the IP header (e.g., TTL) are excluded from this calculation.
- **Confidentiality:** AH **does not offer encryption**, so the content (payload) of the IP packet remains in cleartext.

2. ESP (Encapsulating Security Payload) - Confidentiality, Integrity, and Authenticity:

- **Purpose:** It is the most versatile and common protocol. It provides **confidentiality** (encryption) and optionally **integrity** and **authenticity**.
- **Mechanism:** Encrypts the **payload** and encapsulates the whole package, adding an ESP header before the payload and an ESP trailer at the end.
- **Combined Use:** In practice, ESP is almost always configured to provide both encryption and authentication, making it superior to AH for comprehensive data protection.

Operational Modes (Configurations)

IPsec can be configured to operate in two distinct modes, which differ in what parts are protected and how the packet is encapsulated:

- **Transport Mode:**
 - **Protection:** Only the **payload** (Layer 4 data and above) of the IP packet is protected (encrypted and/or authenticated).
 - **IP Header:** The original IP header remains **in cleartext** and is used for routing.
 - **Usage:** Used for **end-to-end** (host-to-host) protection when both endpoints are IPsec-compatible and reside on the same network, or when one only needs to hide the application payload.
- **Tunnel Mode:**
 - **Protection:** The **entire original IP packet** (header + payload) is protected (encrypted and/or authenticated).
 - **Encapsulation:** The protected packet is **encapsulated** inside a **new external IP header**.
 - **Usage:** The core foundation of **VPNs** (Virtual Private Networks). It is used to create a protected tunnel between two network gateways (site-to-site) or between a host and a gateway (remote-access). It hides the internal (original) IP addresses within the encrypted packet, preventing identification of the two communicating users.

4.2 TLS (Transport Layer Security) - Layer 4/5 (Transport)

- **Definition:** TLS is the cryptographic protocol successor to SSL (Secure Sockets Layer), designed to provide communications security over computer networks.
- **Function:** It provides confidentiality (encryption), integrity (MAC), and authentication between two applications (e.g., a browser and a web server) communicating over the TCP protocol.
- **Main Phases (Handshake):**
 1. **Hello and Negotiation (Session Initiation):**
 - **Client Hello:** The client initiates the communication by sending a **Client Hello** message that includes its supported TLS versions, a list of preferred cryptographic algorithms (Cipher Suites), and a **client random number** (Nonce).
 - **Server Hello:** The server responds with a **Server Hello** message selecting the TLS version and the Cipher Suite it will use, and sends its own **server random number**.
 2. **Authentication and Key Exchange:**
 - **Certificate and Asymmetric Key Exchange:** The server sends its **Digital Certificate** (containing its public key) to the client, which verifies it. Next, the two endpoints use key exchange protocols (e.g., Diffie-Hellman) and asymmetric encryption to calculate the shared **Symmetric Session Key** (Premaster Secret), without it ever being transmitted in cleartext.
 - **Finished:** Both parties send an encrypted message to confirm that the handshake completed successfully and that they are ready for secure communication.
 3. **Data Transmission (Record Protocol):**
 - Once the session is established, application data is split into records, which are then **encrypted** (confidentiality) and authenticated via a **MAC** (integrity) using the agreed symmetric key.
- **Network Layer:** It operates across the **Application Layer** (for the handshake and handling certificates) and the **Transport Layer** (for encapsulation over TCP).
- **Advantages:** Widely supported, essential for the web, and provides robust **end-to-end** session protection.

Cipher Suites: Algorithms and Historical Restrictions

1. Explanation and Structure of the Cipher Suite

A **Cipher Suite** (or Cyphersuite) is a nested set of cryptographic algorithms agreed upon during the TLS/SSL Handshake that defines **how** secure communication will take place. Each suite is composed of four key elements:

Component	Function and Example (Example: TLS_RSA_WITH_AES_128_GCM_SHA256)
Protocol	Indicates the security protocol (e.g., SSL or TLS).
Key Exchange Alg.	Method for exchanging the symmetric session key (e.g., RSA , Diffie-Hellman, ECDHE).
Encryption Alg.	Symmetric algorithm used to encrypt data (e.g., RC4, DES, AES_128_GCM).
Hash/MAC Alg.	Hash function used for integrity and authentication (e.g., MD5, SHA, SHA256).

Figure 2: Structure of a TLS/SSL Cipher Suite.

Cipher Suites and Export Restrictions

Cipher Suites that include **EXPORT** in their name (**SSL_RSA_EXPORT_WITH...**) are a historical relic of the **crypto wars** in the United States and post-war weapon control treaties.

Implication: Initial Inability to Communicate

The issue with negotiating an **EXPORT** suite was not a total inability to communicate, but rather the **deliberate degradation of security** at the start of communication to comply with export laws.

During the TLS/SSL Handshake (negotiation phase), if a client outside the USA connected to a US server, the only **legal shared Cipher Suite** they could agree on was a weak **EXPORT** suite. This left the connection highly vulnerable to interception by government agencies or external attackers.

4.3 HTTPS (Hypertext Transfer Protocol Secure) - Layer 7 (Application)

- **Definition:** HTTPS is **not** a standalone security protocol, but simply the use of standard HTTP (Hypertext Transfer Protocol) **layered over** a **TLS/SSL** security layer.
- **Formula:** The relationship is defined as:

$$\text{HTTPS} = \text{HTTP} + \text{TLS/SSL}$$

- **Port:** It uses the standard port **443**, instead of HTTP's port 80.
- **Function:** All data exchanged between the browser (client) and the web server, including URLs, headers, and form content, is encrypted and authenticated by the underlying TLS layer.
- **Objective:** Protection of **web transactions** and user identity, ensuring that the user is talking to the correct server.

5 End-to-End Encryption and Key Exchange

These sections illustrate two foundational concepts of modern cryptography: ensuring that only final participants read the data (**E2EE**) and how to securely establish a secret channel for communication (**Diffie-Hellman**).

5.1 End-to-End Encryption (E2EE)

End-to-End Encryption (E2EE) is a communication system where **only the communicating users can read the messages**. No third party, including Internet service providers, network administrators, or the company providing the communication service (e.g., the messaging platform), has the ability to access the cleartext content of the messages or files exchanged.

- **Core Principle:** Encryption and decryption occur exclusively on the **end users' devices**. The message is encrypted by the sender and decrypted only by the final recipient.
- **Intermediate State:** All messages traveling between the two endpoints (and stored on intermediate servers) remain in an **encrypted (unreadable) format**.
- **Access Keys:** Service providers **do not possess the private keys** necessary to decrypt the messages. This contrasts with **hop-by-hop** encryption (or transport-layer encryption, like standard TLS between a browser and a server), where the message is decrypted on the server before being re-encrypted for its final destination.

5.2 Diffie-Hellman (DH) Key Exchange Algorithm

The Diffie-Hellman algorithm is a fundamental cryptographic method invented in 1976 that allows two parties who have never interacted before and who are communicating over an **unsecured channel** to jointly establish a shared **symmetric secret key**.

Core Principle: One-Way Functions and the Discrete Logarithm Problem

The effectiveness of DH relies on a mathematical property known as the **Discrete Logarithm Problem**. It is easy to perform exponentiation (the "forward" direction), but it is extremely difficult and computationally expensive to perform the inverse operation (the discrete logarithm) to find the original exponent.

Phases of the Algorithm (The "Color" Exchange)

Let's assume Alice and Bob want to agree on a common secret color, starting from a public color known to everyone (the base).

1. **Public Choice (Base):** Alice and Bob agree on a large **public prime number** (p) and a **public primitive root** (g). These values are known to anyone, including an eavesdropper.
2. **Secret Selection:**
 - Alice chooses a **random secret number** (a).
 - Bob chooses a **random secret number** (b).
3. **Exchange of Partial Public Values:**
 - Alice calculates and sends to Bob the value A :

$$A = g^a$$

- Bob calculates and sends to Alice the value B :

$$B = g^b$$

(An eavesdropper intercepts A and B , but does not know a or b).

4. Calculation of the Shared Secret Key (K):

- Alice uses Bob's value (B) with her secret key (a):

$$K = B^a = (g^b)^a = g^{ab}$$

- Bob uses Alice's value (A) with his secret key (b):

$$K = A^b = (g^a)^b = g^{ab}$$

Since g^{ab} is identical for both, **Alice and Bob have obtained the same secret key K** without Eve being able to determine it (as she would need to solve the discrete logarithm to find a or b from A or B). The key K is then used for high-speed **symmetric encryption** of the session data.

6 Modern Protocols for Secure Messaging

6.1 1. MTPROTO (Mobile Telegram Protocol)

MTPROTO is the proprietary cryptographic protocol developed specifically for the **Telegram** messaging application. It is designed to be optimized for mobile connections and to handle large amounts of data efficiently.

- **Structure:** MTPROTO is divided into three layers:
 1. **High-Level Layer (API):** Defines the methods and objects for API queries and communication between client and server.
 2. **Cryptographic Layer:** Handles authorization, authentication via RSA keys, and Diffie-Hellman key exchanges to establish an AES session key.
 3. **Transport Layer:** Provides data transport (often over TCP or HTTP).
- **Encryption Modes:**
 - **Standard Chats/Groups (Cloud Chat):** Messages are encrypted **client-server/server-client**. Data is encrypted between your device and the Telegram server, but **resides encrypted on the servers** (enabling sync across multiple devices). This is not End-to-End.
 - **Secret Chats:** These use **End-to-End Encryption (E2EE)**. Messages are not stored on the server and are accessible only on the participating devices. They support self-destruction and are tied to a single device.
WARNING: we will see that in reality, even this is not strictly End-to-End
- **Criticisms:** Being a proprietary and non-standardized protocol, it has faced criticisms for its complexity and the lack of deep review by the external cryptographic community, although no critical vulnerabilities have been found.

6.2 2. Signal Protocol (Formerly TextSecure)

The Signal Protocol (developed by Open Whisper Systems and currently used by the Signal app) is widely considered the **gold standard** for communications security and End-to-End Encryption (E2EE).

- **Structure and Purpose:** Designed to be asynchronous (works even offline) and to provide strong E2EE for messages, voice calls, and video.
- **Key Technologies:**
 - **Double Ratchet Algorithm (DMA):** Uses a combination of Diffie-Hellman handshakes and hash functions to continuously generate new symmetric keys for **every single message** (**MTPROTO refreshes every 100 messages**).
 - **Forward Secrecy:** If a session key is compromised, only future messages are exposed, because past keys are completely wiped out (forward because moving forward, historical messages stay protected).
 - **Future Secrecy (Break-in Recovery):** If a current key is compromised, future generated keys automatically recover protection.
- **Implementation:** The protocol is completely **open source** and subject to continuous auditing by expert cryptographers, which is why it has been adopted by many other platforms like WhatsApp, Facebook Messenger, and Google Messages.

7 Comparative Analysis of Messaging Platforms

The security of an application depends not only on its protocol, but also on its implementation, default settings, and policies regarding metadata collection.

7.1 Protocols and Encryption (Confidentiality Focus)

App	Encryption Protocol	Default E2EE
Signal	Signal Protocol	Yes (1:1 and Groups)
WhatsApp	Signal Protocol	Yes (1:1 and Groups)
Telegram	MTPROTO	No (Secret Chats Only)
iMessage	Apple Proprietary	Yes (Between Apple Users)
Snapchat	TLS + Proprietary	Partial (Calls/Video Only)
Instagram (DM)	Transport (TLS)	No E2EE
TikTok (DM)	Transport (TLS)	No E2EE

7.2 Privacy and Metadata (Privacy Focus)

App	Privacy Level	Metadata Collection / Info Exposed
Signal	Maximum	Minimal (Only phone number and last registration timestamp).
WhatsApp	Excellent	High (IP, contacts, device ID, who you talk to. Used by Meta for telemetry/analytics).
Telegram	Variable	High (IP, Contacts, device ID, contact sync. Required for Cloud functionality).
iMessage	Good	Medium (Metadata and backups managed by Apple).
Snapchat	Medium	High (Location, contacts, interactions, demographic data for advertising).
Instagram (DM)	Low	Maximum (Interactions, interests, advertising tracking tied to Meta ecosystem).
TikTok (DM)	Low	Maximum (Interests, viewing metrics, network data used for the algorithm).

7.3 Confidentiality Analysis: Practical Limits of E2EE and Functional Backdoors

The security provided by End-to-End Encryption (E2EE) is theoretically robust, but its practical **confidentiality (privacy)** is often compromised by convenience features and design choices that create potential access points for the provider.

1. Default E2EE Settings and Cloud Features:

- **Telegram (Cloud Chat):** It is crucial to reiterate that Telegram uses Client-Server encryption, not E2EE, for standard chats. Messages are stored on the servers. For **Secret Chats** (E2EE), Telegram uses its own key-exchange implementation. (**Medium, depends on implementation**)

2. Leaks and Compromise Points in WhatsApp (Meta):

Despite using the robust Signal Protocol, WhatsApp features exceptions that limit absolute E2EE coverage:

- **Cloud Backups (External Decryption):** For user convenience, cloud backups on Google Drive or iCloud **are not end-to-end encrypted by default** (though WhatsApp has introduced an explicit option for E2EE backups). If the user does not enable this, message copies are decrypted and re-encrypted using keys managed by cloud providers (Google/Apple), making them accessible to third parties. (**Insecure**)

- **Web/Desktop Access (On-Device Decryption):** Features like WhatsApp Web or the Desktop app require messages to be **decrypted on the primary mobile device** and then **transmitted via a separate secure channel** to the secondary client. (**Secure**)
- **Message Previews (Decrypted Metadata):** To display link previews (e.g., the title of a shared article), WhatsApp must send the URL to **Meta's servers** (or a caching service) to parse info (title, image). While message text stays encrypted, the URL (a sensitive metadata element revealing context) is **temporarily decrypted or exposed to the servers**. (**Mid, only shared links**)

3. Metadata Collection (What They Know About Us):

- **Metadata (All Apps):** Metadata (who speaks to whom, when, from where, on what device) **is never end-to-end encrypted**. Companies like **Meta** (WhatsApp) and **ByteDance** (TikTok) collect highly detailed metadata on user behavior and relationships, utilizing it for advertising targeting and social graph analysis, even if they cannot read E2EE message content. (**Insecure**)
- **Signal (Minimal Collection):** Collects only the bare minimum required for operations (phone number and last connection day), drastically limiting the privacy attack surface. (**Secure**)

Absolute **confidentiality** requires not just a strong E2EE protocol, but also the removal of features like unencrypted cloud backups, loose multi-device sync, and total absence of metadata analytics.